

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



ابزار دقیق

تمرین سری سوم

نام و نام خانوادگی:

امیرعلی دهقانی

شماره دانشجویی:

۸۱۰۱۰۲۴۴۳

فهرست مطالب

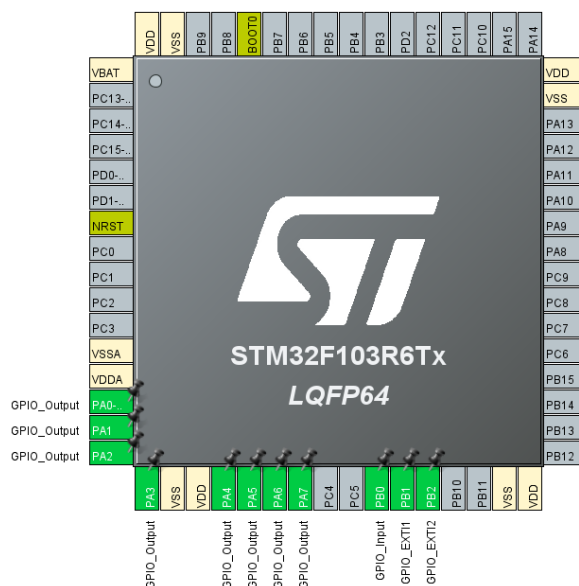
سوال ۱	۲
سوال ۲	۶
سوال ۳	۹
سوال ۴	۱۱

سوال ۱

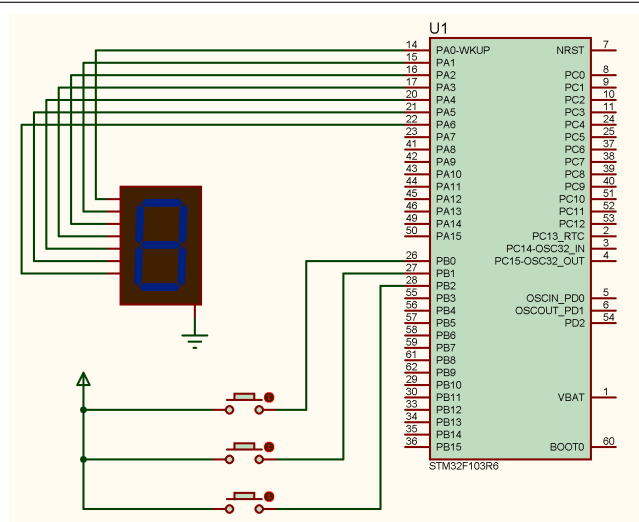
توجه: برای این سوال به اشتباه از میکروکنترلر STM32F103R6 استفاده کردم که این موضوع با چیفتی‌ای درس هماهنگ شد.

الف: تنظیم پایه‌ها و شماتیک مدار

پایه‌های PA0 تا PA6 میکروکنترلر به عنوان خروجی تنظیم شده و به پایه‌های A تا G سون‌سگمنت کاتد مشترک متصل شدند. پایه PB0 به عنوان ورودی دیجیتال برای کلید اول و پایه‌های PB1 و PB2 به عنوان وقفه خارجی (External Interrupt) برای کلیدهای دوم و سوم تنظیم شدند.



شکل ۱: تنظیم پایه‌های میکروکنترلر در نرم‌افزار STM32CubeIDE



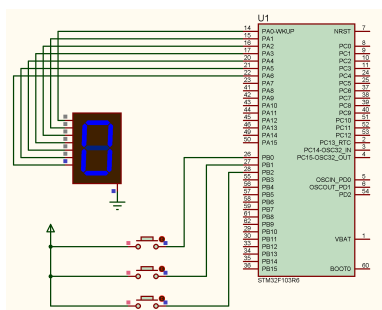
شکل ۲: شماتیک مدار شبیه‌سازی شده در نرم‌افزار Proteus

ب: نمایش عدد صفر در حالت اولیه

برای روشن شدن هر سگمنت باید ولتاژ منطقی ۱ (SET) و برای خاموش ماندن آن ولتاژ منطقی ۰ (RESET) اعمال شود. جهت نمایش عدد ۰، سگمنت‌های A تا F باید روشن و سگمنت (G) باید خاموش باشد. بنابراین تنظیمات در کد پیش از شروع حلقه به این صورت انجام می‌شود:

PA0, PA1, PA2, PA3, PA4, PA5 : 1

PA6 : 0



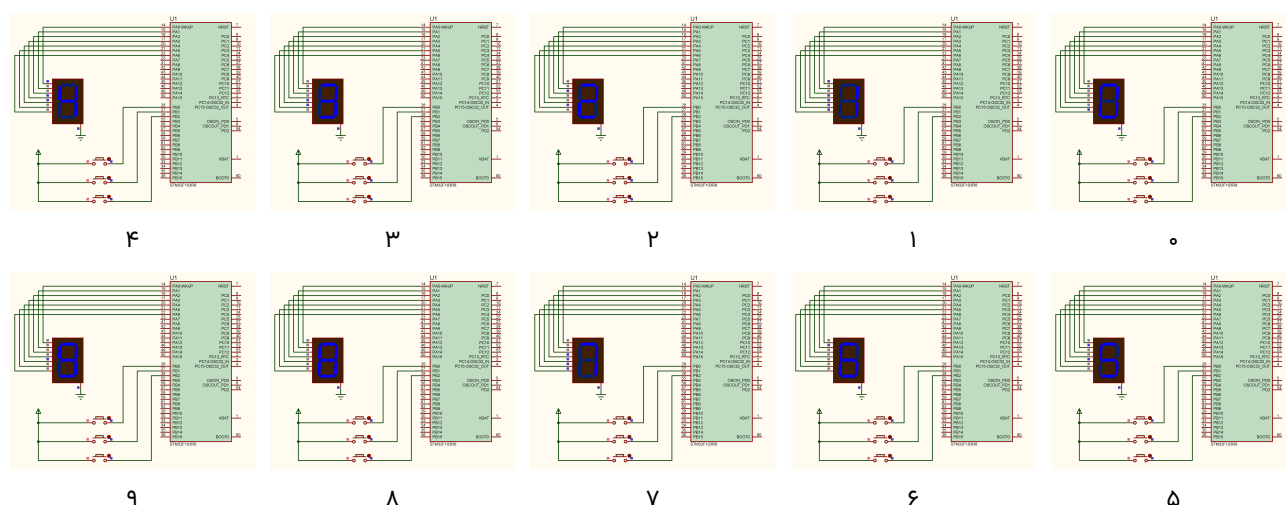
شکل ۳: نمایش عدد صفر در وضعیت اولیه در Proteus

ج: پیاده‌سازی شمارش ۱ تا ۹

برای این بخش در ابتدا جهت سهولت، تمامی کدهای هگزادسیمال اعداد ۰ تا ۹ برای روشن شدن سون سگمنت در آرایه segCode به صورت گلوبال تعریف شد. همچنین متغیر isRunning که درواقع flag برای این است که دکمه اول فشرده شده و باید سیستم شروع به کار بکند تعریف شد.

منطق کلی کد به این صورت است که با فشرده شدن کلید متصل به پایه PB0، isRunning = ۱ می‌شود و در حلقه بی‌نهایت برنامه، هرسی این شرط چک می‌شود که اگر مقدار این متغیر ۱ بود (درواقع یعنی دکمه فشار داده شده بود)، در یک حلقه for به ترتیب حالت‌های پایه‌های خروجی با مقادیر آرایه segCode هرسی برابر شوند که با متصل کردن آن به سون سگمنت، دقیقا اعداد بین ۰ تا ۹ به ترتیب نمایش داده می‌شوند. همچنین برای فاصله بین نمایش اعداد روی سون سگمنت هم مقدار Delay

برابر با $1s = 1000ms$ در نظر گرفته شد.



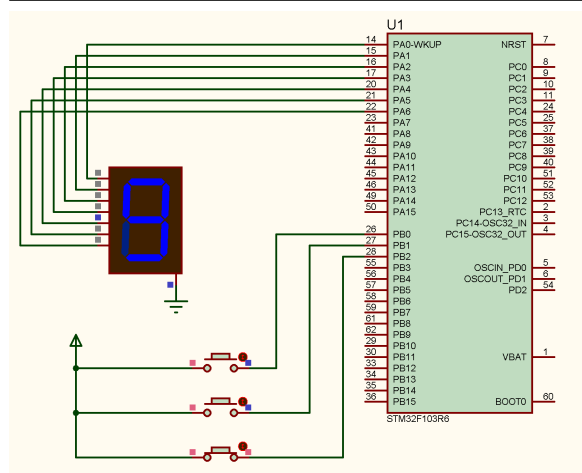
شکل ۴: مراحل نمایش اعداد از ۰ تا ۹ در شبیه‌سازی پروتئوس

د: پیاده‌سازی متوقف‌کننده‌ها

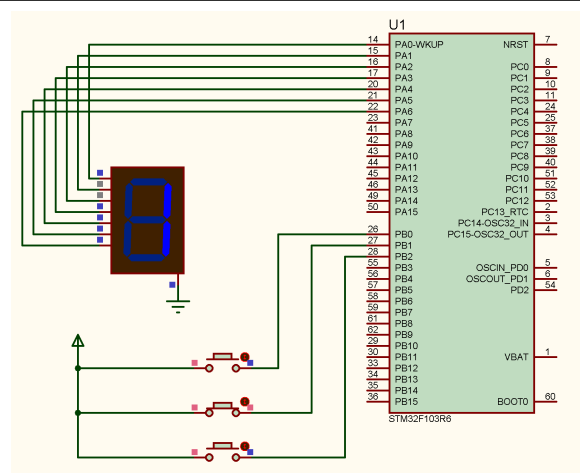
برای پیاده‌سازی این بخش و جلوگیری از تداخل وقفه‌ها با روند عادی برنامه، متغیر `systemState` به عنوان یک متغیر `Global` و به صورت `volatile` تعریف شد تا تغییرات آن درون توابع `Interrupt` بلافاصله برای حلقه اصلی برنامه قابل تشخیص باشد. متغیر `currentNumber` نیز صرفاً به عنوان شمارنده در حلقه اصلی تعریف شد.

منطق کار به این شکل است که برنامه در ابتدا با وضعیت ۰، عدد صفر را نشان می‌دهد و منتظر فشردن کلید اول می‌ماند. با فشردن این کلید، وضعیت به ۱ تغییر کرده و روند شمارش آغاز می‌شود. به جای استفاده از حلقه `for` (که در صورت وقوع وقفه ممکن است بعد از بازگشت به کار خود ادامه دهد)، از افزایش متغیر `currentNumber` درون حلقه `while` استفاده کردیم. سیستم پس از نمایش هر عدد و اعمال تاخیر ۱ ثانیه‌ای، مجدداً وضعیت را چک می‌کند تا فقط در صورت عدم تغییر وضعیت، شمارنده را افزایش دهد.

با فشردن کلید دوم، تابع وقفه بلافاصله اجرا شده و مقدار `systemState` را برابر ۲ قرار می‌دهد. در این حالت، حلقه اصلی برنامه مستقیماً عدد ۱ را از آرایه `segCode` خوانده و روی سون‌سگمنت نمایش می‌دهد. از آنجا که در این کد هیچ شرطی برای بازگشت از وضعیت ۲ تعریف نشده است، سیستم در همین حالت کاملاً متوقف می‌ماند. به طریق مشابه، با فشردن کلید سوم `PB2`، وضعیت به ۳ تغییر کرده و سیستم به طور دائم عدد ۹ را نمایش داده و متوقف می‌شود. این دو حالت فقط با فشردن دیگر کلید وقفه، تغییر می‌کنند.



(ب) فشردن کلید سوم؛ توقف و نمایش عدد ۹



(آ) فشردن کلید دوم؛ توقف و نمایش عدد ۱

شکل ۵: عملکرد متوقف‌کننده‌های اضطراری آسانسور

ویدیو کامل مدار شبیه‌سازی این سوال در فولدر video قرار دارد.

سوال ۲

الف: واحد TIM و تولید سیگنال PWM

واحد TIM در میکروکنترلرهای STM32 برای کارهایی مثل زمان سنجی، تولید سیگنال‌های PWM، اندازه‌گیری پالس و ایجاد وقفه‌های زمان‌بندی شده استفاده می‌شود. برای تولید یک سیگنال PWM، دو رجیستر زیر تنظیم می‌شوند:

• **Prescaler (PSC):** این رجیستر فرکانس کلاک ورودی به تایمر را تقسیم کرده و کاهش می‌دهد تا سرعت شمارش تایمر به میزان دلخواه برسد.

• **Auto-Reload Register (ARR):** این رجیستر حداکثر مقداری که تایمر تا آن می‌شمارد را تعیین می‌کند. تایمر از صفر تا این مقدار شمرده و سپس ریست می‌شود تا یک دوره تناوب شکل بگیرد.

رابطه ریاضی برای محاسبه فرکانس سیگنال PWM:

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1) \times (ARR + 1)}$$

فرکانس کلاک میکروکنترلر ۳۶MHz است. برای تولید سیگنال PWM با فرکانس ۱kHz، باید مقادیر PSC و ARR به گونه‌ای انتخاب شوند که حاصل ضرب مخرج برابر ۳۶۰۰۰ شود:

$$1000 = \frac{36000000}{(PSC + 1) \times (ARR + 1)} \Rightarrow (PSC + 1) \times (ARR + 1) = 36000$$

بنابراین، مقادیر زیر برای تایمر تنظیم شده است:

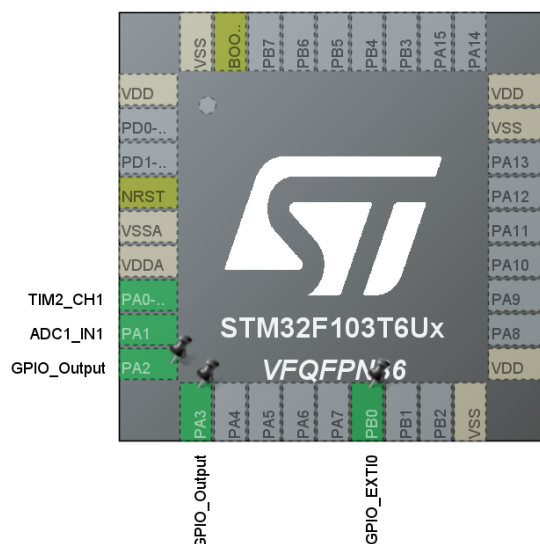
$$PSC = 35 \quad | \quad ARR = 999$$

ب: برنامه‌نویسی برای کنترل سرعت و جهت موتور

از آنجایی که پایه‌های میکروکنترلر توانایی تامین جریان کافی برای چرخاندن مستقیم موتور را ندارند، برای اتصال موتور به سیستم از درایور L298 استفاده شد. در این مدار، پایه‌های IN1 و IN2 درایور به ترتیب به پایه‌های خروجی PA2 و PA3 میکروکنترلر متصل شدند تا جهت چرخش موتور کنترل شود.

برای مدیریت جهت حرکت، یک متغیر گلوبال به نام motorDirection تعریف شد. کلید تغییر جهت نیز به پایه PB0 متصل شد و به صورت Interrupt تنظیم شد. با هر بار فشردن این کلید، تابع وقفه اجرا شده و مقدار متغیر motorDirection را برعکس می‌کند (از صفر به یک و بالعکس). در حلقه اصلی برنامه، سیستم مدام این متغیر را بررسی می‌کند؛ اگر مقدار آن صفر باشد، PA2 روشن و PA3 خاموش می‌شود و در غیر این صورت، جای آن‌ها عوض می‌شود تا موتور در جهت معکوس بچرخد.

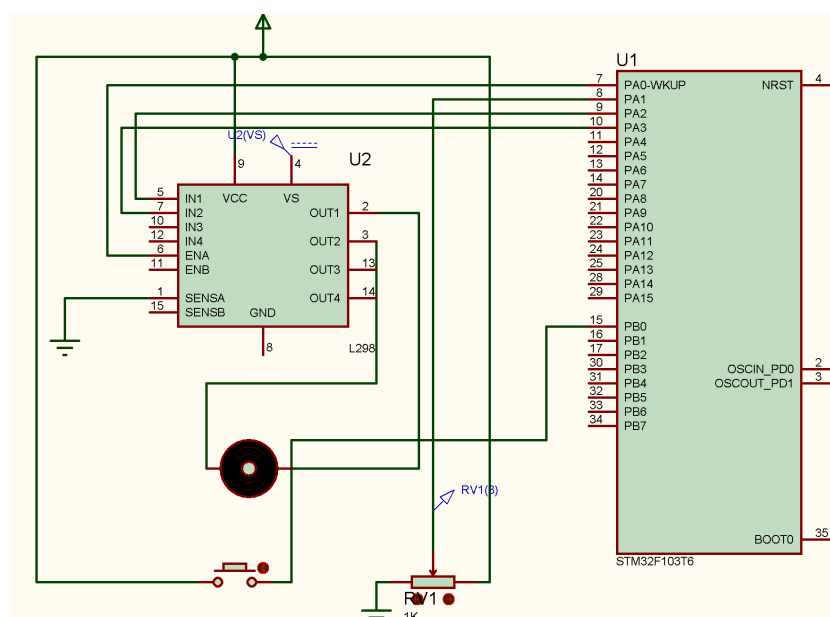
همچنین برای کنترل سرعت موتور، از واحد مبدل آنالوگ به دیجیتال (ADC) استفاده شد. پتانسیومتر به پایه PA1 متصل شد. از آنجا که واحد ADC در این میکروکنترلر ۱۲ بیتی است، خواندن ولتاژ پتانسیومتر عددی بین ۰ تا ۴۰۹۵ تولید می‌کند که این عدد باید به بازه ۰ تا ۱۰۰۰ (که همان سقف شمارش یا ARR تایمر در بخش الف است) تبدیل شود. سپس این عدد جدید مستقیماً به تایمر داده می‌شود تا عرض پالس سیگنال PWM (یا همان Duty Cycle) تغییر کند. با این کار، سرعت موتور به صورت نرم و هماهنگ با چرخاندن پتانسیومتر کم و زیاد می‌شود.



شکل ۶: میکروکنترلر STM32F103T6 تنظیم شده در نرم افزار

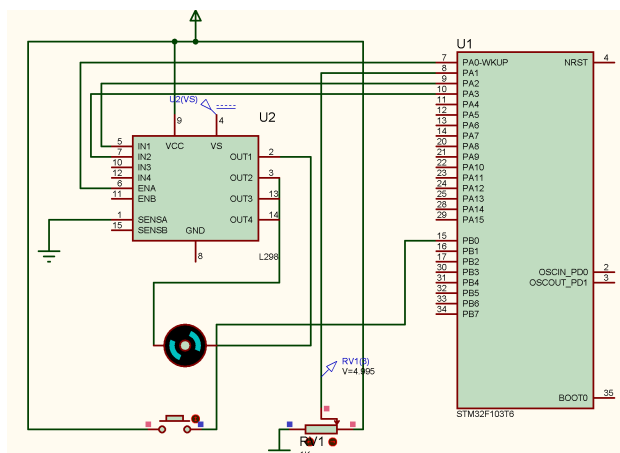
ج: شبیه‌سازی مدار در پروتئوس

در این بخش، مدار طراحی شده را در نرم‌افزار پروتئوس پیاده‌سازی شد. برای راه‌اندازی موتور از درایور L298 استفاده شد. پایه ENA درایور که وظیفه کنترل سرعت را بر عهده دارد، به خروجی PWM میکروکنترلر (پایه PA0) متصل گردید تا سیگنال تولید شده مستقیماً به آن وارد شود. پایه‌های IN1 و IN2 نیز برای تعیین جهت چرخش، به ترتیب به پایه‌های PA2 و PA3 وصل شدند. همان‌طور که در نکات شبیه‌سازی ذکر شده بود، برای عملکرد صحیح درایور و روشن شدن موتور، پایه SEN_A (حسگر جریان موتور) مستقیماً به زمین (GND) متصل شد. همچنین برای تامین توان کافی جهت چرخش موتور با سرعت بالا، تغذیه اصلی موتور (پایه VS) از تغذیه منطقی مدار جدا شده و به یک منبع ولتاژ DC مجزا با ولتاژ بالاتر متصل گردید. پتانسیومتر نیز به عنوان مقسم ولتاژ در مدار قرار گرفت و پایه متغیر آن به ورودی ADC (پایه PA1) وصل شد.

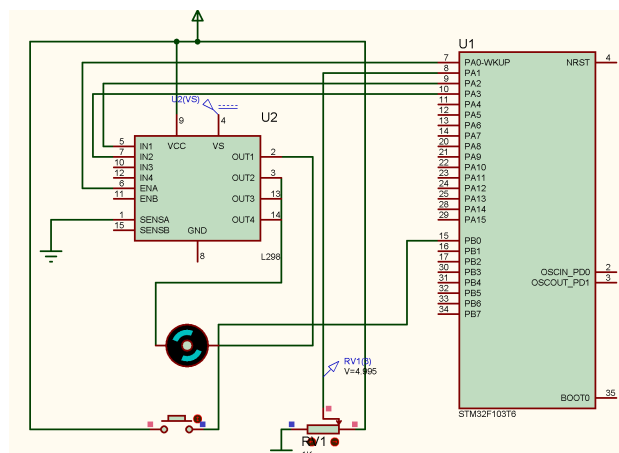


شکل ۷: شماتیک کلی اتصالات مدار کنترل موتور و درایور L298 در پروتئوس

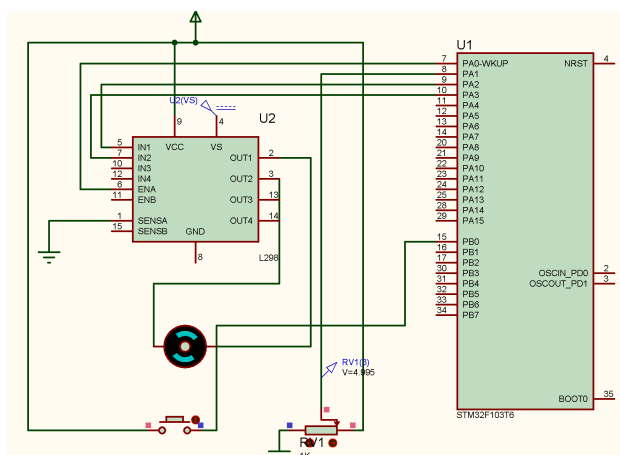
همچنین جهت بررسی دقیق‌تر عملکرد مدار، ویدیوی کامل شبیه‌سازی در فولدر video قرار داده شده است. در شکل‌های زیر، تست عملکرد سیستم به صورت خلاصه آورده شده است:



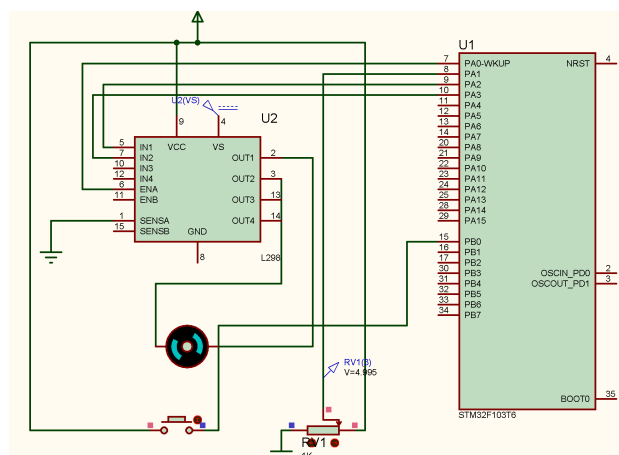
(ب) چرخش در جهت ساعت‌گرد



(آ) حالت اولیه



(د) چرخش در جهت پادساعت‌گرد



(ج) فشردن دکمه و تغییر جهت

شکل ۸: بررسی عملکرد سیستم و تغییر جهت چرخش موتور

سوال ۳

طراحی سیستم شمارش معکوس با استفاده از وقفه تایمر در STM32

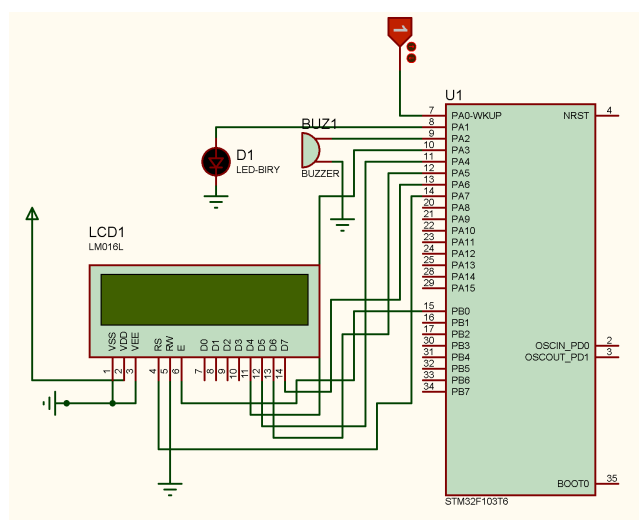
در این بخش، یک سیستم شمارش معکوس ۱۵ ثانیه‌ای با نمایشگر LCD، باز و ال‌ای‌دی پیاده‌سازی شد. **منطق برنامه‌نویسی و کنترل سخت‌افزار:** برای ایجاد زمان‌بندی دقیق، تایمر میکروکنترلر به گونه‌ای تنظیم شد که دقیقاً هر ۵۰۰ میلی‌ثانیه (نیم ثانیه) یک وقفه تولید کند. با تغییر وضعیت کلید Logicstate (متصل به پایه PA0) به حالت یک، متغیرهای کنترلی فعال شده و روند شمارش معکوس آغاز می‌شود.

در تابع وقفه (Callback)، یک شمارنده کمکی قرار داده شده است که هر نیم ثانیه یک واحد افزایش می‌یابد. با استفاده از این شمارنده (و عملگر باقیمانده)، خروجی‌های ال‌ای‌دی و باز در هر نیم ثانیه تغییر وضعیت می‌دهند (خاموش و روشن می‌شوند) تا حالت چشمک‌زن و بوق مقطع دقیقاً در هر ثانیه ایجاد شود. پس از گذشت دو نیم‌ثانیه (یک ثانیه کامل)، یک واحد از زمان اصلی کاسته می‌شود.

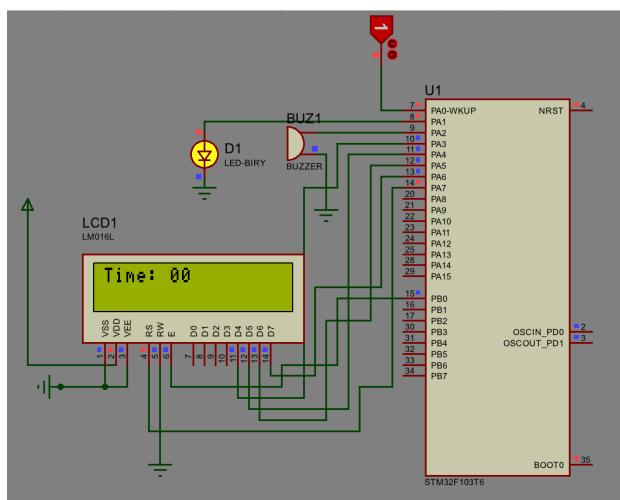
طبق خواسته سوال، هنگامی که شمارشگر به صفر می‌رسد، روند کاهش زمان متوقف شده و ال‌ای‌دی و باز به صورت ممتد روشن می‌مانند. همچنین اگر در هر لحظه از شبیه‌سازی، کلید ورودی روی حالت صفر قرار گیرد، برنامه بلافاصله زمان را به ۱۵ ریست کرده، خروجی‌ها را خاموش می‌کند و منتظر فرمان مجدد می‌ماند.

اتصالات مدار در پروتئوس: برای راه‌اندازی LCD کاراکتری در حالت ۴ بیتی، پایه‌های دیتای آن (D4 تا D7) به پایه‌های PA3 تا PA6 میکروکنترلر متصل شدند. پایه‌های کنترلی RS و E نیز به ترتیب به PA7 و PB0 وصل شدند. برای رفع مشکلات شبیه‌سازی، پایه‌های تغذیه نمایشگر (VDD و VSS) مستقیماً به منابع تغذیه و زمین متصل گردیدند. ولتاژ کاری باز نیز اصلاح شد تا با ولتاژ منطقی میکروکنترلر همخوانی داشته باشد.

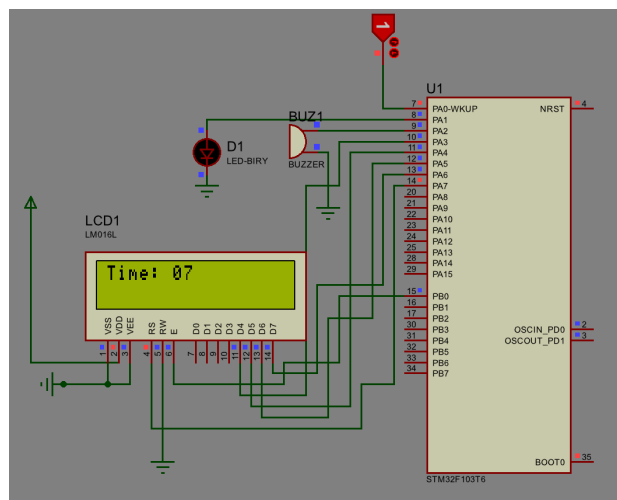
جهت مشاهده کامل روند کارکرد سیستم (شامل بوق زدن مقطع، چشمک زدن ال‌ای‌دی، توقف روی صفر و ریست شدن)، ویدیوی شبیه‌سازی به صورت کامل ضبط شده و در فولدر video قرار داده شده است. تصاویر مربوط به شماتیک کلی مدار و وضعیت‌های مختلف سیستم نیز در ادامه آورده شده‌اند:



شکل ۹: شماتیک کلی اتصالات میکروکنترلر، LCD، باز و ال‌ای‌دی در محیط پروتئوس



(ب) پایان شمارش و روشن ماندن ممتد خروجی‌ها



(آ) وضعیت سیستم در حین شمارش معکوس

شکل ۱۰: بررسی عملکرد سیستم در حالت‌های مختلف

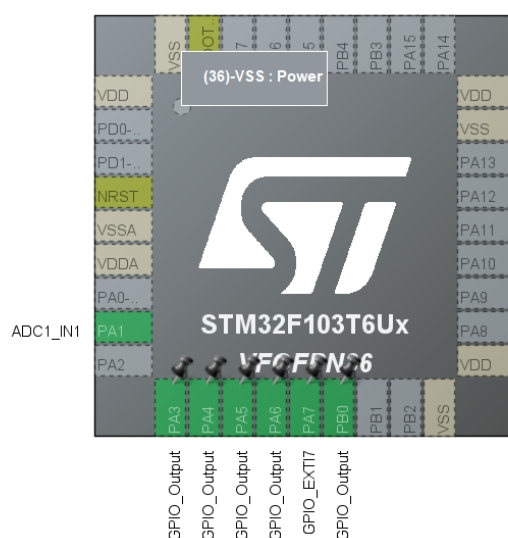
سوال ۴

الف: بررسی مبدل آنالوگ به دیجیتال (ADC)

ADC وظیفه دارد ولتاژهای پیوسته و فیزیکی تولید شده توسط سنسورها (یا پتانسیومتر در این شبیه‌سازی) را دریافت کرده و به اعداد دیجیتال تبدیل کند تا برای پردازنده میکروکنترلر قابل فهم باشد. برای اضافه کردن آن در نرم‌افزار STM32CubeIDE، از منوی تنظیمات وارد بخش Analog شده و واحد ADC1 را انتخاب می‌کنیم. سپس با تیک زدن کانال ورودی اول (IN1)، این واحد فعال می‌شود.

رزولوشن این مبدل ۱۲ بیت تعیین شده است. دلیل اول این انتخاب محدودیت سخت‌افزاری است؛ چرا که واحد ADC در خانواده میکروکنترلرهای STM32F1 (مانند میکروکنترلر استفاده شده در این پروژه) به صورت سخت‌افزاری ۱۲ بیتی طراحی شده است. دلیل دوم، دقت بسیار بالای این رزولوشن است. با یک مبدل ۱۲ بیتی، بازه ولتاژ ۰ تا ۳.۳ ولت به $4096 = 2^{12}$ پله (اعداد ۰ تا ۴۰۹۵) تقسیم می‌شود. این امر به میکروکنترلر اجازه می‌دهد تا تغییرات ولتاژ را با دقت بسیار بالایی (حدود ۰/۸ میلی‌ولت) تشخیص دهد که برای خواندن مقادیر دقیق سنسورهای صنعتی فوق‌العاده است.

در نهایت، با فعال‌سازی کانال IN1 در تنظیمات نرم‌افزار، پایه PA1 به طور خودکار به عنوان ورودی آنالوگ تعیین شد تا ولتاژ تولید شده توسط سنسور از طریق این پایه خوانده و وارد واحد ADC شود.



شکل ۱۱: تنظیم پایه‌های میکروکنترلر در نرم‌افزار STM32CubeIDE

ب: برنامه‌نویسی جهت نمونه‌برداری پیوسته و محاسبه ولتاژ

در این بخش، کدهای مربوط به خواندن مقادیر آنالوگ و تبدیل آن‌ها به ولتاژ واقعی در محیط STM32CubeIDE نوشته شد. از آنجا که رزولوشن مبدل ADC میکروکنترلر ۱۲ بیتی است، خروجی دیجیتال آن همواره عددی صحیح در بازه ۰ تا ۴۰۹۵ خواهد بود. در حلقه اصلی برنامه (while)، ابتدا با استفاده از دستور HAL_ADC_Start فرآیند نمونه‌برداری آغاز شده و پس از اطمینان از اتمام تبدیل با تابع HAL_ADC_PollForConversion، مقدار دیجیتال خوانده شده در متغیر adcValue ذخیره می‌شود. سپس با استفاده از تناسب زیر و با در نظر گرفتن ولتاژ مرجع ۳.۳ ولت، ولتاژ لحظه‌ای سنسور محاسبه می‌شود:

$$\text{Voltage} = \frac{\text{ADC_Value} \times 3.3}{4095}$$

این مقدار در متغیر sensorVoltage از نوع float ذخیره می‌گردد.

ج: تشخیص نوع سنسور و محاسبه مقدار فیزیکی

در این بخش، با استفاده از ساختار شرطی، بازه ولتاژ محاسبه شده در مرحله قبل بررسی می‌شود تا نوع سنسور متصل شده به صورت خودکار تشخیص داده شود. سپس بر اساس فرمول‌های ارائه شده در جدول صورت سوال، مقدار فیزیکی متناظر محاسبه می‌گردد.

متغیرهای sensorName و unit برای ذخیره نام سنسور و واحد اندازه‌گیری، و متغیر physicalValue از نوع float برای ذخیره مقدار نهایی تعریف شده‌اند. منطق پیاده‌سازی شده به شرح زیر است:

• ولتاژ ۰.۰ تا ۹۹.۰ ولت: سنسور دمای LM35 تشخیص داده شده و مقدار دما به سانتی‌گراد از رابطه $\text{Voltage} \times 100$ به دست می‌آید.

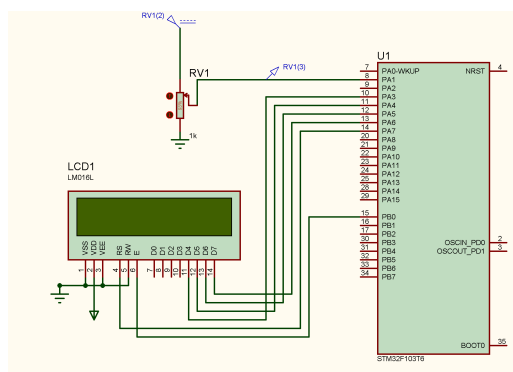
• ولتاژ ۰.۱ تا ۹۹.۱ ولت: سنسور رطوبت HIH-4000 تشخیص داده شده و درصد رطوبت با رابطه $\text{Voltage} \times 33.3$ محاسبه می‌گردد.

• ولتاژ ۰.۲ تا ۹۹.۲ ولت: سنسور فشار MPX4115 شناسایی شده و فشار بر حسب هکتوپاسکال از رابطه $\text{Voltage} \times 100$ محاسبه می‌شود.

• ولتاژ ۰.۳ تا ۳۰.۳ ولت: سنسور کیفیت هوای MQ-135 تشخیص داده شده و درصد کیفیت هوا از رابطه $(\text{Voltage} / 3.3) \times 100$ به دست می‌آید.

د: شبیه‌سازی مدار در Proteus نمایش اطلاعات بر روی LCD

برای پیاده‌سازی این سیستم در محیط پروتئوس، از یک پتانسیومتر اکتیو به عنوان شبیه‌ساز سنسور استفاده شد. پایه متغیر این پتانسیومتر به کانال اول مبدل آنالوگ به دیجیتال میکروکنترلر (پایه PA1) متصل گردید. نکته مهم در این بخش، تنظیم دقیق ولتاژ مرجع مبدل بر روی ۳.۳ ولت از طریق تنظیمات پروتئوس بود تا محاسبات میکروکنترلر تطابق کاملی با مقادیر آنالوگ ورودی داشته باشد. همچنین اتصالات نمایشگر LCD در حالت ۴ بیتی مشابه سوال قبل انجام شد. در شکل زیر، شماتیک کلی و پایه‌های متصل شده در نرم‌افزار پروتئوس قابل مشاهده است:

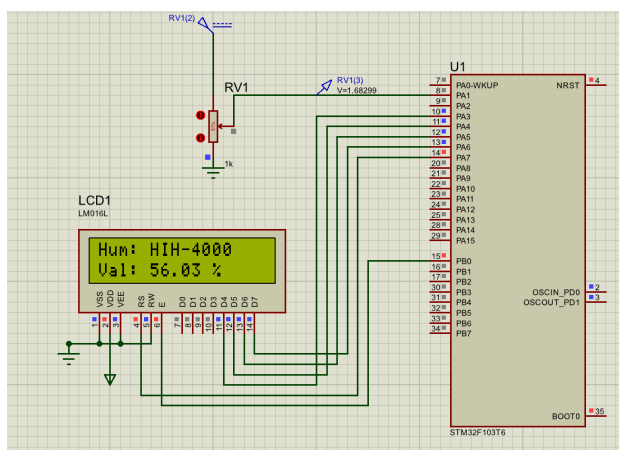


شکل ۱۲: شماتیک کلی اتصالات مبدل آنالوگ به دیجیتال و نمایشگر در پروتئوس

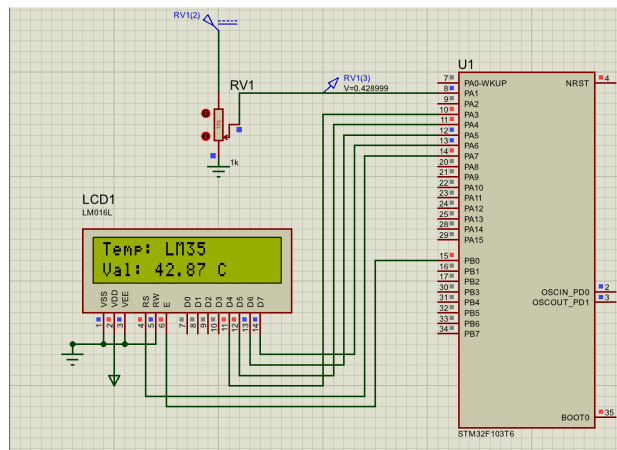
در گام نهایی، مقادیر پردازش شده باید در یک قالب مشخص بر روی نمایشگر چاپ می‌شدند. برای این منظور، از توابع استاندارد stdio.h و دستور printf استفاده شد.

مطابق صورت سوال، در خط اول نمایشگر نام سنسور تشخیص داده شده و در خط دوم مقدار فیزیکی محاسبه شده و واحد اندازه‌گیری آن چاپ گردید.

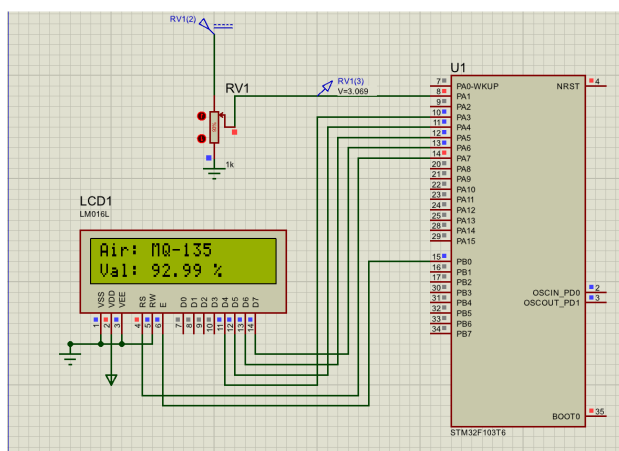
با تغییر ولتاژ پتانسیومتر از ۰ تا ۳.۳ ولت، میکروکنترلر به صورت پیوسته تغییرات را احساس کرده و نوع سنسور را تغییر می‌دهد. خروجی موفقیت‌آمیز شبیه‌سازی برای هر چهار بازه ولتاژی و چهار سنسور مختلف (دما، رطوبت، فشار و کیفیت هوا) در قالب تصاویر زیر آورده شده است:



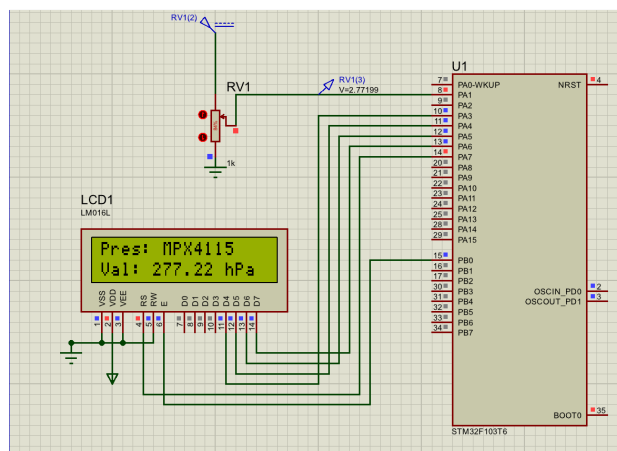
(ب) سنسور رطوبت (HIH-4000) در بازه ولتاژ ۱/۹۹ تا ۱/۰



(ت) سنسور دما (LM35) در بازه ولتاژ ۰/۰ تا ۰/۹۹



(د) سنسور کیفیت هوا (MQ-135) در بازه ولتاژ ۳/۰ تا ۳/۳



(ج) سنسور فشار (MPX4115) در بازه ولتاژ ۲/۰ تا ۲/۹۹

شکل ۱۳: بررسی عملکرد سیستم تشخیص خودکار سنسور و نمایش مقادیر فیزیکی

همچنین ویدیو کامل شبیه‌سازی این سوال در فولدر video قرار دارد.